
RAPPORT TECHNIQUE

ESLint

Lintier statique pour JavaScript & TypeScript

Université Africaine des Sciences — Gabon
Année académique 2025–2026

Présenté par :

- MBINA-MASSOUSA STEPHY JORDAN GABY
- MVE NDONG KYSS LEMIEN

Sous la supervision de :

- Dr. AFFANE MOUNDOUNGA ANSELME R.

1. Introduction

ESLint est un outil d'analyse statique du code source JavaScript et TypeScript, libre et open source. Il permet d'identifier et de corriger les problèmes dans le code avant son exécution, en appliquant un ensemble de règles configurables par le développeur ou l'équipe.

Créé en 2013 par Nicholas C. Zakas, ESLint est aujourd'hui maintenu par la OpenJS Foundation et constitue l'outil de linting le plus utilisé dans l'écosystème JavaScript, avec plus de 44 millions de téléchargements hebdomadaires sur npm.

Objectifs de ce rapport :

- Comprendre le fonctionnement d'ESLint et son architecture
- Présenter l'installation, la configuration et les règles essentielles
- Illustrer concrètement son impact via des exemples de code Avant / Après
- Passer en revue l'écosystème de plugins et les bonnes pratiques

2. Contexte et historique

2.1 Naissance du linting JavaScript

Avant l'apparition d'ESLint, les développeurs JavaScript disposaient principalement de JSLint (Douglas Crockford, 2002) et JSHint (fork communautaire, 2011). Ces outils souffraient d'un manque de flexibilité : les règles étaient difficilement personnalisables et l'architecture ne permettait pas l'ajout de règles tierces.

Nicholas C. Zakas a créé ESLint en 2013 pour répondre à ces limitations, en s'appuyant sur un modèle à base d'AST (Abstract Syntax Tree) permettant une analyse fine du code et une extensibilité totale via un système de plugins.

2.2 Évolution et adoption

ESLint est devenu le standard de facto pour le linting JavaScript grâce à :

- Sa capacité à analyser ES6+, JSX, TypeScript via des parseurs interchangeables
- Son modèle de plugins qui permet à l'écosystème de contribuer des règles spécialisées
- Son intégration native dans tous les IDE majeurs (VS Code, WebStorm, Vim...)
- Son adoption par des frameworks majeurs : React, Vue, Angular, Node.js

Année	Version	Événement clé
2013	v0.0.1	Première version publique par Nicholas C. Zakas
2015	v1.0	Support ES6, plugins, règles personnalisées
2016	v3.0	Abandon du support IE8, amélioration des performances
2019	v6.0	Support ESLint pour ESMODULES, amélioration TypeScript
2023	v8.0	Annonce de la Flat Config (eslint.config.js)
2024	v9.0	Flat Config devient le standard officiel

3. Architecture et fonctionnement

3.1 Analyse statique et AST

ESLint repose sur l'analyse statique du code, c'est-à-dire qu'il examine le code source sans l'exécuter. Le cœur du processus est la transformation du code en AST (Abstract Syntax Tree), une représentation arborescente de la structure syntaxique du programme.

Par défaut, ESLint utilise Espree comme parseur, qui est compatible avec les spécifications ECMAScript modernes (ES2022+). Il est possible de remplacer ce parseur par :

- `@typescript-eslint/parser` — pour TypeScript
- `@babel/eslint-parser` — pour du code Babel non standard
- `vue-eslint-parser` — pour les fichiers `.vue` (Single File Components)

3.2 Pipeline d'exécution

Voici les quatre étapes du pipeline ESLint lors de l'analyse d'un fichier :

1. Parsing : Le code source est lu et transformé en AST par le parseur
2. Traversal : ESLint traverse l'AST noeud par noeud (visitor pattern)
3. Rule check : Chaque règle activée inspecte les noeuds qui la concernent
4. Report : Les violations sont collectées, formatées et affichées à l'utilisateur

Exemple : la règle `eqeqeq` s'abonne au noeud AST de type `BinaryExpression`.
A chaque fois qu'ESLint rencontre une expression `==` ou `!=`, la règle est appelée et peut signaler une erreur si l'opérateur n'est pas `===` ou `!==`.

3.3 Architecture modulaire

L'architecture d'ESLint est entièrement modulaire :

- Parseur : interchangeable (Espree, `@typescript-eslint/parser`, etc.)
- Règles : chaque règle est un module JS indépendant, exportant des callbacks AST
- Formateurs : la sortie peut être formatée en JSON, HTML, table, stylish...
- Plugins : regroupent règles, parseurs et configurations pour un écosystème donné

4. Installation et configuration

4.1 Installation

ESLint s'installe via npm ou yarn dans un projet Node.js existant :

```
# Installation avec configuration automatique (recommande)
npm init @eslint/config@latest

# Installation manuelle
npm install --save-dev eslint
```

```
npx eslint --init
```

La commande `npm init @eslint/config@latest` lance un assistant interactif qui détecte automatiquement le framework utilisé (React, Vue, etc.) et génère un fichier de configuration adapté.

4.2 Fichier de configuration (Flat Config, v9+)

Depuis ESLint v9, le format de configuration standard est le Flat Config, défini dans un fichier `eslint.config.js` ou `eslint.config.mjs` à la racine du projet :

```
// eslint.config.js
import js from '@eslint/js';

export default [
  js.configs.recommended,
  {
    rules: {
      "no-unused-vars": "error",
      "semi": ["warn", "always"],
      "eqeqeq": "error",
      "no-console": "warn",
      "no-var": "error"
    },
    languageOptions: { ecmaVersion: "latest" }
  }
];
```

Les trois niveaux de sévérité d'une règle sont :

- "error" (ou 2) — bloque le build, code de sortie 1
- "warn" (ou 1) — avertissement, code de sortie 0
- "off" (ou 0) — règle désactivée

4.3 Commandes essentielles

```
# Analyser un fichier ou un dossier
npx eslint src/

# Corriger automatiquement les erreurs réparables
npx eslint src/ --fix

# Cibler des extensions spécifiques
npx eslint src/ --ext .js,.ts,.jsx

# Ignorer des fichiers ou dossiers
# Dans eslint.config.js : { ignores: ['dist/', 'node_modules/'] }
```

5. Règles essentielles

ESLint est livré avec plusieurs centaines de règles intégrées. Voici les plus couramment utilisées, regroupées par catégorie :

5.1 Qualité du code

Règle	Sévérité	Description
<code>no-unused-vars</code>	error	Interdit les variables déclarées mais jamais utilisées
<code>no-undef</code>	error	Interdit l'utilisation de variables non déclarées
<code>eqeqeq</code>	error	Impose l'utilisation de <code>===</code> et <code>!==</code> (au lieu de <code>==</code> et <code>!=</code>)
<code>no-var</code>	error	Interdit le mot-cle <code>var</code> , préfère <code>let</code> et <code>const</code>
<code>prefer-const</code>	warn	Préfère <code>const</code> lorsque la variable n'est pas réassignée
<code>no-empty</code>	warn	Interdit les blocs de code vides <code>{}</code>

5.2 Style et formatage

Règle	Sévérité	Description
<code>semi</code>	warn	Exige ou interdit les points-virgules en fin d'instruction
<code>curly</code>	warn	Exige des accolades pour tous les blocs (<code>if</code> , <code>for</code> , <code>while</code> ...)
<code>arrow-spacing</code>	warn	Exige un espace avant et après <code>=></code> dans les arrow functions
<code>no-console</code>	warn	Avertit sur les appels <code>console.log/error/warn</code>
<code>indent</code>	warn	Impose une indentation cohérente (2 ou 4 espaces)
<code>quotes</code>	warn	Impose l'usage de simples ou doubles guillemets de façon cohérente

5.3 Bonnes pratiques

Règle	Sévérité	Description
<code>no-eval</code>	error	Interdit l'usage de <code>eval()</code> (sécurité)
<code>no-alert</code>	warn	Interdit <code>alert()</code> , <code>confirm()</code> , <code>prompt()</code> dans le code de prod
<code>no-duplicate-imports</code>	error	Interdit les imports dupliqués depuis le même module

Règle	Sévérité	Description
<code>no-shadow</code>	warn	Evite la re-declaration de variables dans des scopes internes
<code>consistent-return</code>	warn	Impose un retour de valeur coherent dans toutes les branches

6. Illustrations : code avant et après ESLint

Les exemples suivants illustrent l'impact concret d'ESLint sur la qualité du code. Chaque exemple montre le code original avec ses violations, puis le code corrigé après application des règles ESLint.

6.1 Exemple 1 : variables et opérateurs de comparaison

Règles impliquées : no-unused-vars, no-var, eqeqeq, semi

AVANT (avec erreurs)	APRÈS (code valide)
<pre>var username = 'Alice' var age = 25 // erreur: var var unused = 'oubliee' // erreur: non utilisee function checkAge(a) { if (a == 18) { // erreur: == au lieu de === console.log('adulte') // warn: pas de ; } } checkAge(age)</pre> 4 erreurs ESLint detectees	<pre>const username = 'Alice' // OK: const const age = 25 // OK: const // OK: supprimee function checkAge(a) { if (a === 18) { // OK: === console.log('adulte'); // OK: ; } } checkAge(age);</pre> 0 erreur — code valide

6.2 Exemple 2 : arrow functions et accolades

Règles impliquées : arrow-spacing, no-console, curly, prefer-const, no-undef

AVANT (avec erreurs)	APRÈS (code valide)
<pre>console.log(undeclaredVar) // erreur: no-undef let multiply = (x,y)=>{ // erreur: arrow-spacing console.log('calcul') // warn: no-console return x * y } let result = multiply(3,4) // erreur: let->const</pre>	<pre>const declaredVar = 42; // OK: declare const multiply = (x, y) => { // OK: spacing // console supprime // OK return x * y; }; const result = multiply(3,4); // OK: const if (result > 10) { // OK: accolades alert('grand'); // OK: curly }</pre> 0 erreur — code valide

AVANT (avec erreurs)	APRÈS (code valide)
<pre>if (result > 10) // erreur: pas d'accolades console.log('grand') // warn: curly</pre> 5 erreurs / avertissements	

6.3 Sortie terminal typique

Voici un exemple de rapport ESLint affiché dans le terminal :

```
$ npx eslint src/app.js

/project/src/app.js
  2:5  error  'unused' is defined but never used    no-unused-vars
  6:10 error  Expected '===' and instead saw '=='   eqeqeq
  7:23 warning Unexpected console statement    no-console
  9:1  error  'var' declarations are not allowed    no-var
 12:3  error  Missing semicolon                    semi

x  5 problems (4 errors, 1 warning)
   4 errors and 0 warnings potentially fixable with --fix
```

7. Plugins et intégrations

7.1 Plugins populaires

L'écosystème ESLint est très riche. Les plugins permettent d'étendre les règles disponibles pour des frameworks ou des environnements spécifiques :

Plugin	Description
eslint-plugin-react	Règles pour les composants React et JSX
@typescript-eslint/eslint-plugin	Règles TypeScript avec analyse de types
eslint-plugin-vue	Règles pour les Single File Components Vue.js
eslint-plugin-import	Contrôle des imports/exports ES6 modules
eslint-config-airbnb	Guide de style Airbnb, très utilisé en entreprise
eslint-plugin-prettier	Intègre Prettier comme règle ESLint
eslint-plugin-node	Règles spécifiques à l'environnement Node.js
eslint-plugin-security	Détecte les failles de sécurité JavaScript courantes

7.2 Intégration dans les IDE

- VS Code : extension ESLint (dbaeumer.vscode-eslint) — affichage inline des erreurs
- WebStorm / IntelliJ : intégré nativement, aucune configuration supplémentaire

- Vim / Neovim : via ALE ou nvim-lint avec configuration LSP
- Emacs : flycheck-eslint pour l'affichage en temps réel

7.3 Intégration CI/CD

ESLint s'intègre facilement dans les pipelines d'intégration continue. Exemple avec GitHub Actions :

```
# .github/workflows/lint.yml
name: ESLint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm ci
      - run: npx eslint src/ --max-warnings 0
```

8. Bonnes pratiques

8.1 Organisation de la configuration

- Utiliser une configuration partagée (eslint-config-monprojet) pour uniformiser plusieurs dépôts
- Basé sa config sur les règles recommandées : js.configs.recommended comme point de départ
- Documenter chaque règle désactivée avec un commentaire justificatif
- Séparer les configurations par environnement : browser, node, test

8.2 Workflow en équipe

- Intégrer ESLint dans le hook pre-commit via Husky et lint-staged
- Activer ESLint dans le pipeline CI avec --max-warnings 0 pour bloquer les PRs non conformes
- Combiner avec Prettier pour séparer les préoccupations : Prettier formate, ESLint détecte les bugs
- Utiliser eslint-disable-next-line de manière exceptionnelle et toujours justifiée

8.3 Performance

- Utiliser .eslintignore ou la propriété ignores pour exclure dist/, node_modules/, *.min.js
- Activer le cache ESLint avec --cache pour les projets volumineux
- Limiter les plugins aux seuls nécessaires pour réduire le temps d'analyse

Commande recommandée pour la CI :

```
npx eslint src/ --cache --cache-location .eslintcache --max-warnings 0
```


Cette commande échoue si la moindre erreur ou le moindre avertissement est détecté, garantissant ainsi un code conforme avant chaque merge.

9. Conclusion

ESLint s'est imposé comme un pilier incontournable du développement JavaScript moderne. Au-delà de la simple détection d'erreurs syntaxiques, il constitue un véritable outil de gouvernance du code, garantissant la cohérence, la maintenabilité et la sécurité des projets au sein d'une équipe.

Grâce à son architecture extensible, ses centaines de règles intégrées et son écosystème de plugins, ESLint s'adapte à tout contexte : applications React, API Node.js, projets TypeScript, monorepos, etc.

L'investissement dans la configuration d'ESLint est rapidement rentabilisé : moins de bugs en production, des revues de code plus rapides, et un onboarding facilité pour les nouveaux membres de l'équipe.

Ressources officielles :

- Documentation : <https://eslint.org/docs/latest/>
- Dépôt GitHub : <https://github.com/eslint/eslint>
- Playground : <https://eslint.org/play>
- npm : <https://www.npmjs.com/package/eslint>